SOCIAL EAGLE

Gen AI Architect Program

DEPLOYMENT GUIDE

From your laptop to a live website on your own domain with HTTPS. Written for complete beginners.

LOCAL DOCKER	GITHUB	VPS DEPLOY	DOMAIN + SSL

18 STEPS · COMPLETE COMMAND REFERENCE · BEGINNER LEVEL

What This Guide Covers

You have an application running on your laptop. By the end of this guide it will be running on a real server, reachable at your own domain, with a padlock in the browser. Every command is written out in full, so you can copy and paste them one by one.

Part 1: On Your Laptop

1	Understand the project structure
2	Write the Dockerfile for the backend
3	Write the Dockerfile for the frontend
4	Add .dockerignore files
5	Write docker-compose.yml
6	Test everything locally with Docker Desktop
7	Push the code to GitHub

Part 2: On the Server (VPS / VM)

8	Connect to your VPS over SSH
9	Install Docker and Git
10	Create a GitHub Personal Access Token
11	Clone your repository into the VM
12	Start the containers with Docker Compose
13	Verify with docker ps
14	Point your domain to the VM (DNS A record)
15	Confirm propagation with a DNS checker
16	Install Nginx and write the reverse proxy config
17	Test the config and reload Nginx
18	Install Certbot and enable free SSL

Part 3: Reference

- Updating your app after deployment (the redeploy loop)
- Command cheat sheet
- Common errors and how to fix them

Before You Start

What you need

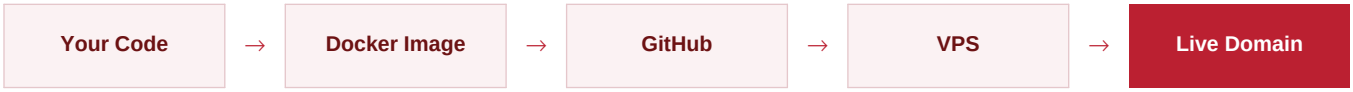
Item	Why you need it	Cost
A working app	A frontend folder and a backend folder that already run on your laptop.	N/A
Docker Desktop	To build and test containers locally before touching the server.	Free
A GitHub account	To store your code and pull it onto the server.	Free
A VPS / VM	The always-on computer that runs your app. Hostinger, DigitalOcean, AWS Lightsail or Contabo. 2 GB RAM is a comfortable minimum.	Paid
A domain name	So people type **yourapp.com** instead of an IP address. GoDaddy, Namecheap, Hostinger.	Paid

The five ideas behind everything you are about to do

Term	In plain English
Image	A frozen snapshot of your app plus everything it needs to run: Python, Node, libraries and your code. Built once, runs anywhere.
Container	A running copy of an image. Like opening a saved file. You can start, stop and delete it without affecting the image.
Dockerfile	The recipe that tells Docker how to build the image. One per application (one for frontend, one for backend).
Docker Compose	A single file that starts several containers together and lets them talk to each other.
Reverse proxy	Nginx sits in front of your containers. Visitors reach Nginx on port 80/443, and Nginx quietly forwards them to the right container.

Why use Docker at all?
Without it, you would install Python, Node, and a dozen libraries by hand on the server, and spend the evening fixing “it works on my machine” bugs. With Docker, the exact environment that worked on your laptop is the one that runs on the server. Nothing drifts.

The journey in one line



Part 1: On Your Laptop

1 Understand the project structure

Goal: know exactly where each file goes before you create it.

Docker needs a clear folder layout. Arrange your project like this. The names **frontend** and **backend** are what we will use throughout the guide. If yours differ, change them consistently everywhere.

PROJECT LAYOUT

```
my-project/
|
|-- backend/
|   |-- app/                  <- your Python code
|   |-- main.py               <- FastAPI / Flask entry point
|   |-- requirements.txt      <- Python dependencies
|   |-- Dockerfile            <- YOU WILL CREATE THIS (Step 2)
|   |-- .dockerignore         <- YOU WILL CREATE THIS (Step 4)
|   \-- .env                  <- secrets, NEVER pushed to GitHub
|
|-- frontend/
|   |-- src/                  <- your React code
|   |-- public/
|   |-- package.json
|   |-- nginx.conf            <- YOU WILL CREATE THIS (Step 3)
|   |-- Dockerfile            <- YOU WILL CREATE THIS (Step 3)
|   \-- .dockerignore         <- YOU WILL CREATE THIS (Step 4)
|
|-- docker-compose.yml        <- YOU WILL CREATE THIS (Step 5)
\-- .gitignore                <- YOU WILL CREATE THIS (Step 7)
```

The .env file must never reach GitHub

Your API keys, database passwords and tokens live in **.env**. If you push it to a public repository, bots will find and abuse those keys within minutes. Step 7 shows you how to block it with .gitignore. Keep a copy of the file safely, because you will recreate it manually on the server in Step 11.

2 Write the Dockerfile for the backend

Goal: one recipe file that packages your API into an image.

Create a file named exactly **Dockerfile** (capital D, no file extension) inside the **backend** folder. Paste the following:

backend/Dockerfile (FastAPI example)

```
# 1. Start from a small official Python image
FROM python:3.11-slim

# 2. All following commands run inside this folder
WORKDIR /app

# 3. Copy ONLY requirements first - this makes rebuilds much faster
COPY requirements.txt .

# 4. Install the Python packages
RUN pip install --no-cache-dir -r requirements.txt

# 5. Now copy the rest of the source code
COPY . .

# 6. Document which port the app listens on
EXPOSE 8000

# 7. The command that starts the app when the container runs
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

0.0.0.0 is not optional

If you start your app on **127.0.0.1**, it will only accept connections from inside the container, and your browser will get "connection refused" even though the container looks healthy. Always bind to **0.0.0.0** inside Docker.

Using Flask instead of FastAPI?

Replace line 7 with the Gunicorn command below and add **gunicorn** to your requirements.txt.

backend/Dockerfile (Flask alternative)

```
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "app:app"]
```

What each instruction actually does

Instruction	Meaning
`FROM`	The starting base image. **-slim** variants are far smaller than the full image, so builds and uploads are quicker.
`WORKDIR`	Creates and moves into a folder inside the container. Everything after this happens there.
`COPY`	Copies files from your laptop into the image.
`RUN`	Executes a command *while building* the image. The result is baked in permanently.
`EXPOSE`	Documentation for humans and tooling. It does not publish the port by itself. Compose does that.
`CMD`	The command run *when the container starts*. Only the last CMD in a file counts.

3 Write the Dockerfile for the frontend

Goal: build the React app, then serve the finished files with a tiny web server.

A React app is not “run” in production. It is **built** into plain HTML, CSS and JavaScript files, which a small web server then hands out. We do both in one file using a **multi-stage build**: stage one compiles, stage two serves. The heavy Node tooling is thrown away, so the final image is around 25 MB instead of 400 MB.

frontend/Dockerfile

```
# ---------- STAGE 1: build the React app ----------
FROM node:20-alpine AS build
WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# ---------- STAGE 2: serve the built files ----------
FROM nginx:alpine

# Vite outputs to /dist. Create React App outputs to /build
COPY --from=build /app/dist /usr/share/nginx/html

# Our own config so page refresh does not 404
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

dist or build: check which one you have
Vite projects output to **/app/dist**. **Create React App** outputs to **/app/build**. If you copy the wrong folder the build fails with “not found”. Run **npm run build** once on your laptop and look at which folder appears.

The nginx.conf that goes with it

Create **nginx.conf** in the **frontend** folder. Without this, refreshing the page on any route other than the home page returns a 404, because the server looks for a file that does not exist.

frontend/nginx.conf

```
server {
    listen 80;
    server_name localhost;

    root /usr/share/nginx/html;
    index index.html;

    # Send every unknown path back to index.html
    # so React Router can handle it
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

4 Add .dockerignore files

Goal: keep junk out of your images so builds stay fast.

Without these, Docker copies **node_modules** and your entire Git history into the image, turning a 30-second build into a 10-minute one. Create one file in each folder.

frontend/.dockerignore

```
node_modules
dist
build
.git
.env
*.log
.DS_Store
```

backend/.dockerignore

```
__pycache__
*.pyc
venv
.venv
.git
.env
*.log
```

5 Write docker-compose.yml

Goal: one file that starts both containers together with a single command.

Create **docker-compose.yml** in the **root** of your project (next to the frontend and backend folders, not inside them). Indentation in YAML is meaningful. Use **two spaces**, never tabs.

docker-compose.yml

```
services:

  backend:
    build: ./backend
    container_name: my-backend
    ports:
      - "8000:8000"        # host port : container port
    env_file:
      - ./backend/.env
    restart: unless-stopped
    networks:
      - app-network

  frontend:
    build: ./frontend
    container_name: my-frontend
    ports:
      - "3000:80"          # visit localhost:3000 in the browser
    depends_on:
      - backend
    restart: unless-stopped
    networks:
      - app-network

networks:
  app-network:
    driver: bridge
```

Reading the important lines

Line	What it means
`build: ./backend`	Look in that folder for a Dockerfile and build the image from it.
`ports: "8000:8000"`	Left = port on the host machine. Right = port inside the container. Change only the left number if 8000 is already busy.
`env_file`	Loads your secrets into the container at run time. The file stays outside the image, which is exactly what you want.
`restart: unless-stopped`	If the server reboots or the app crashes, Docker starts it again automatically. Essential in production.
`depends_on`	Starts the backend first. Note: it waits for the container to *start*, not for the app inside to be ready.
`networks`	Puts both containers on a shared private network so they can find each other by name.

How the frontend should call the backend

Inside the shared network, the backend is reachable at **http://backend:8000**, because the service name works as a hostname. But your React code runs in the visitor's browser, outside that network, so it must call the public address instead. Use a relative path like **/api/users** and let Nginx route it (Step 16). That way the same code works locally and in production.

6 Test everything locally with Docker Desktop

Goal: prove it works on your laptop before you touch a server.

Open Docker Desktop and wait until the whale icon in your system tray stops animating. That means the engine is running. Then open a terminal in your project root.

TERMINAL · your project folder

```
# Build the images and start both containers
docker compose up --build

# The first build takes several minutes. Later builds are much faster
# because Docker reuses cached layers.
```

Now check it in the browser:

Open this	You should see
`http://localhost:3000`	Your frontend, looking exactly as it does in development.
`http://localhost:8000/docs`	The FastAPI interactive documentation page (skip if you use Flask).

Useful commands while testing

TERMINAL

```
docker compose up --build -d   # run in background (detached)
docker compose ps              # what is running right now
docker compose logs -f         # live logs from all services
docker compose logs backend    # logs from one service only
docker compose down            # stop and remove the containers
docker compose down -v         # also delete the volumes (data!)
```

Do not move on until this step passes
Every problem is ten times harder to diagnose over SSH on a remote server. If something is broken, fix it here, on your own machine, where you have your editor and your browser dev tools.

7 Push the code to GitHub

Goal: get your project into a repository the server can clone.

First, protect your secrets

Create a **.gitignore** file in the project root *before* your first commit. Once a secret is committed it stays in the history even if you delete the file later.

.gitignore

```
# Secrets - the most important lines in this file
.env
*.env
!.env.example

# Dependencies
node_modules/
venv/
__pycache__/

# Build output
dist/
build/

# Editor and OS files
.vscode/
.idea/
.DS_Store
```

Then create the repository

Go to **github.com** → **New repository**. Give it a name, choose Public or Private, and do **not** tick “Add a README” because an empty repository is easier to push into. Copy the URL it shows you.

TERMINAL

```
cd my-project

git init
git add .

# Check the list - if .env appears here, STOP and fix .gitignore
git status

git commit -m "Initial commit: dockerised frontend and backend"
git branch -M main
git remote add origin https://github.com/USERNAME/REPO-NAME.git
git push -u origin main
```

Add a .env.example instead
Commit a file called **.env.example** containing the *names* of your variables with dummy values (**OPENAI_API_KEY=your_key_here**). Anyone cloning the repo then knows exactly which variables to set, and no real secret is ever exposed.

Part 2: On the Server

Everything from here runs on your VPS. Your provider gave you three things when you bought it: an **IP address**, a **username** (usually root or ubuntu), and a **password** or SSH key. Keep them in front of you.

8 Connect to your VPS over SSH

Goal: get a terminal on the remote machine.

SSH gives you a command line on the server, as if you were sitting in front of it. On Windows use PowerShell or Windows Terminal; on macOS and Linux use Terminal.

TERMINAL · your laptop

```
ssh root@203.0.113.45

# Replace 203.0.113.45 with YOUR server IP.
# First time it asks 'Are you sure you want to continue connecting?'
# Type: yes

# If your provider gave you a key file instead of a password:
ssh -i /path/to/key.pem ubuntu@203.0.113.45
```

The password is invisible while you type it
Linux does not print asterisks. It looks like nothing is happening, but type the password anyway and press Enter. Better still, paste it.

Update the system first

TERMINAL · on the VPS

```
sudo apt update && sudo apt upgrade -y
```

9 Install Docker and Git

Goal: give the server the same two tools you used locally.

These commands are for **Ubuntu / Debian**, which is what almost every VPS runs. Run them one block at a time and read the output before continuing.

TERMINAL · on the VPS

```
# --- Git (usually already present) ---
sudo apt install git -y
git --version

# --- Docker, using the official convenience script ---
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# --- Let your user run docker without typing sudo every time ---
sudo usermod -aG docker $USER

# --- Start Docker now, and again automatically on every reboot ---
sudo systemctl start docker
sudo systemctl enable docker

# --- Confirm both are installed ---
docker --version
docker compose version
```

Log out and back in after usermod

The group change only takes effect on a new session. Type **exit**, then SSH in again. If you skip this you will keep seeing “permission denied while trying to connect to the Docker daemon socket”.

10 Create a GitHub Personal Access Token

Goal: a password-like key that lets the server clone your repository.

GitHub stopped accepting account passwords over HTTPS in 2021. Instead you generate a **Personal Access Token** (PAT), a long string you use in place of a password.

Click path, exactly as it appears on GitHub

#	Where to click
1	Your profile photo, top right → **Settings**
2	Scroll to the very bottom of the left sidebar → **Developer settings**
3	**Personal access tokens** → **Fine-grained tokens**
4	**Generate new token**
5	**Token name:** anything memorable, e.g. *vps-deploy*
6	**Expiration:** 90 days is a sensible balance
7	**Repository access:** choose *Only select repositories* and pick just the one you are deploying
8	**Permissions** → **Repository permissions** → find **Contents** → set it to **Read and write**
9	**Generate token**, then copy the string that appears

You will only see the token once
The moment you leave that page it is hidden forever. Paste it somewhere safe right now: a password manager, not a chat message. If you lose it, delete the token and generate a new one; there is no way to reveal it again.

What a token looks like

EXAMPLE TOKEN

```
github_pat_11ABCDE0Y0abcdefgh1234_iJkLmNoPqRsTuVwXyZ0123456789AbCd

# Yours will be different. Treat it exactly like a password:
# anyone holding it can read and write your repository.
```

11 Clone your repository into the VM

Goal: get your code onto the server.

TERMINAL · on the VPS

```
# Work from your home folder
cd ~

# Public repository - no token needed
git clone https://github.com/USERNAME/REPO-NAME.git

# Private repository - paste the token in place of the password
git clone https://github.com/USERNAME/REPO-NAME.git
#   Username: your-github-username
#   Password: <paste your token here>

cd REPO-NAME
ls -la          # confirm docker-compose.yml is present
```

Recreate your .env file

Remember, **.env** was deliberately excluded from Git. The server does not have it, and your backend will crash without it. Create it now with the **nano** editor.

TERMINAL · on the VPS

```
nano backend/.env

# Type or paste your variables, for example:
#   OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxx
#   DATABASE_URL=postgresql://user:pass@db:5432/mydb

# To save and exit nano:
#   Ctrl + O   then Enter   (write out)
#   Ctrl + X                (exit)
```

12 Start the containers with Docker Compose

Goal: your app running on the server.

TERMINAL · on the VPS

```
# Make sure you are inside the project folder
cd ~/REPO-NAME

# Build the images and start everything in the background
docker compose up --build -d
```

The flags, decoded

Flag	Meaning
`--build`	Build the images from your Dockerfiles first. Use it every time your code has changed.
`-d`	Detached mode. Runs in the background. Without this, closing your SSH session would kill your app.

If the build is killed on a small VPS
A 1 GB server often runs out of memory during **npm run build**, and the process dies with the word **Killed**. Add swap space: **sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile**, then run the build again.

13 Verify with docker ps

Goal: confirm both containers are actually running, not just created.

TERMINAL · on the VPS

```
docker ps
```

You should see two lines, and the STATUS column is the one that matters:

EXPECTED OUTPUT

```
CONTAINER ID   IMAGE             STATUS          PORTS
a1b2c3d4e5f6   repo-frontend     Up 2 minutes    0.0.0.0:3000->80/tcp
f6e5d4c3b2a1   repo-backend      Up 2 minutes    0.0.0.0:8000->8000/tcp
```

STATUS shows	What it means	What to do
Up 2 minutes	Healthy and running.	Move to Step 14.
Restarting	The app crashes on startup and Docker keeps retrying.	Read the logs. Usually a missing .env variable.
Exited (1)	The app stopped with an error.	Read the logs.
Nothing listed	No container is running at all.	Run **docker ps -a** to see stopped ones, then read their logs.

TERMINAL · diagnosing

```
docker ps -a                    # include stopped containers
docker compose logs -f          # follow all logs, Ctrl+C to quit
docker compose logs backend     # one service only
docker compose restart          # restart everything
```

Test before you involve DNS

Open **http://YOUR_SERVER_IP:3000** in your browser. If your app loads, the containers are fine and everything from here is purely networking. If it does not, check your provider's firewall panel. Many providers block all ports except 22, 80 and 443 by default.

14 Point your domain to the VM

Goal: make yourdomain.com resolve to your server's IP address.

DNS is the internet's phone book. An **A record** is one entry in it: “this name belongs to this IP address”. Log in to wherever you bought your domain (GoDaddy, Namecheap, Hostinger or Cloudflare) and find **DNS Management** or **DNS Zone Editor**.

Add these two records

Type	Name / Host	Value / Points to	TTL
A	@	203.0.113.45	3600
A	**www**	203.0.113.45	3600

Field	What to enter
Type	Always **A** for a normal IPv4 address. AAAA is for IPv6, CNAME points to another name rather than an IP.
Name / Host	@ means the bare domain (yourdomain.com). **www** creates www.yourdomain.com. Add both so visitors reach you either way.
Value	Your server's public IP, copied exactly. Find it with **curl ifconfig.me** on the VPS if you are unsure.
TTL	How long the record is cached, in seconds. 3600 (one hour) is fine. Lower it to 300 while testing if your provider allows.

Delete conflicting records first
Domain registrars often add a default A record pointing at a parking page. If you leave it, half your visitors will land there instead of on your app. Remove any existing A record for @ and **www** before adding yours.

15 Confirm propagation with a DNS checker

Goal: verify the change has spread across the internet before you blame Nginx.

DNS changes are not instant. They ripple outward from server to server. Usually this takes 5 to 30 minutes, occasionally up to 48 hours.

Tool	How to use it
dnschecker.org	Enter your domain, select record type **A**, click Search. You will see a world map of servers. Green ticks mean they already return your IP.
whatsmydns.net	The same idea, presented as a list. Useful for a second opinion.

Or check from any terminal

TERMINAL

```
nslookup yourdomain.com

# or
dig yourdomain.com +short

# Either should print your server IP, e.g. 203.0.113.45
```

Confirm propagation before continuing
Roughly half of “my SSL certificate failed” problems are just impatience. Certbot cannot verify a domain that does not yet point at the server. Get a clear result from the checker first, then move to Step 16.

16 Install Nginx and write the reverse proxy config

Goal: visitors reach yourdomain.com on port 80 instead of IP:3000.

Right now your app answers on port 3000. Nobody types a port number into a browser. Nginx listens on the standard port 80, and forwards each request to the right container behind the scenes.

Install it

TERMINAL · on the VPS

```
sudo apt update
sudo apt install nginx -y

sudo systemctl start nginx
sudo systemctl enable nginx

# Visit http://YOUR_SERVER_IP - you should see
# the default 'Welcome to nginx!' page
```

Create the configuration file

One file per site, named after the domain so you can find it later.

TERMINAL · on the VPS

```
sudo nano /etc/nginx/sites-available/yourdomain.com
```

Paste this, replacing **yourdomain.com** with your real domain:

/etc/nginx/sites-available/yourdomain.com

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;

    # --- Frontend: everything not matched below ---
    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    # --- Backend API: anything starting with /api ---
    location /api/ {
        proxy_pass http://localhost:8000/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Understanding the routing

Directive	What it does
`listen 80`	Answer normal, unencrypted HTTP traffic. Certbot will add a second block for 443 in Step 18.
`server_name`	Which domains this file is responsible for. A request for any other domain will not match here.
`location /`	Matches every URL that no more specific block claims. This is your catch-all for the frontend.
`location /api/`	More specific, so it wins for URLs beginning **/api/**. Everything else still goes to the frontend.
`proxy_pass`	Where to forward the request. This is the container's published port on the host.
`proxy_set_header`	Passes the visitor's real details through. Without these, your backend logs would show every request coming from 127.0.0.1.

The trailing slash in proxy_pass changes the meaning
proxy_pass http://localhost:8000/; (with the trailing slash) strips **/api** before forwarding, so **/api/users** arrives at your backend as **/users**. Without the slash, your backend receives **/api/users** and must define its routes that way. Pick one and be consistent.

17 Test the config and reload Nginx

Goal: activate the site without ever taking it offline.

Enabling a site in Nginx means creating a symbolic link from **sites-available** (where configs live) to **sites-enabled** (what Nginx actually reads).

TERMINAL · on the VPS

```
# 1. Enable the site
sudo ln -s /etc/nginx/sites-available/yourdomain.com \
           /etc/nginx/sites-enabled/

# 2. Remove the default welcome page so it cannot shadow your site
sudo rm /etc/nginx/sites-enabled/default

# 3. TEST the configuration - never skip this
sudo nginx -t

# 4. Only if the test passed, apply the changes
sudo systemctl reload nginx
```

What a passing test looks like

EXPECTED OUTPUT OF nginx -t

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

reload, not restart
reload loads the new configuration while existing connections finish normally, so no visitor notices. **restart** drops every connection. And if you reload a broken config, Nginx refuses and keeps the old one running, which is why **nginx -t** first is such a good habit.

Open the firewall

TERMINAL · on the VPS

```
sudo ufw allow 'Nginx Full'    # opens ports 80 and 443
sudo ufw allow OpenSSH         # do this BEFORE enabling ufw
sudo ufw enable
sudo ufw status
```

Allow OpenSSH before you enable the firewall
If you enable **ufw** without allowing SSH first, you lock yourself out of your own server instantly. Recovery means using your provider's web console. Run the commands in the order shown.

Now visit **http://yourdomain.com**. Your app should load with no port number and no IP address.

18 Install Certbot and enable free SSL

Goal: the padlock icon, and https:// on your domain.

Certbot obtains a free certificate from Let's Encrypt, installs it, and rewrites your Nginx config to serve HTTPS, all automatically. It also sets up renewal, because these certificates expire every 90 days.

TERMINAL · on the VPS

```
# 1. Install Certbot and its Nginx plugin
sudo apt install certbot python3-certbot-nginx -y

# 2. Request the certificate (list every domain you want covered)
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

What Certbot will ask you

Question	What to answer
Enter email address	A real address. Let's Encrypt emails you if renewal ever fails.
Agree to Terms of Service	**Y**
Share email with EFF?	**N** is perfectly fine.
Redirect HTTP to HTTPS?	Choose **2 (Redirect)** so visitors typing http:// are sent to the secure version automatically.

On success you will see **Congratulations! You have successfully enabled HTTPS**. Visit **https://yourdomain.com** and look for the padlock.

Confirm automatic renewal

TERMINAL · on the VPS

```
# Simulate a renewal without actually doing one
sudo certbot renew --dry-run

# Check the timer that runs it twice a day
sudo systemctl status certbot.timer

# List your certificates and their expiry dates
sudo certbot certificates
```

Deployment complete
Your app now runs in containers that restart themselves, behind a reverse proxy, on your own domain, over an encrypted connection that renews itself. This is a standard production deployment.

Part 3: Reference

Updating your app after deployment

This is the loop you will repeat for the rest of the project's life. It takes about a minute.

THE REDEPLOY LOOP

```
# --- On your laptop ---
git add .
git commit -m "Describe what you changed"
git push

# --- On the VPS ---
cd ~/REPO-NAME
git pull
docker compose up --build -d
docker ps                      # confirm both are Up again
```

Nginx and SSL stay untouched

Steps 14 to 18 are one-time setup. Once your domain and certificate are in place, redeploying is only the loop above. You never need to touch Nginx again unless you add a new service or a new port.

Command cheat sheet

Command	What it does
`docker compose up --build -d`	Build images and start everything in the background
`docker compose down`	Stop and remove the containers
`docker compose restart`	Restart without rebuilding
`docker ps`	List running containers
`docker ps -a`	List all containers, including stopped ones
`docker compose logs -f`	Follow live logs (Ctrl+C to exit)
`docker exec -it NAME sh`	Open a shell inside a running container
`docker system prune -a`	Delete unused images and free disk space
`sudo nginx -t`	Test the Nginx configuration for errors
`sudo systemctl reload nginx`	Apply config changes with no downtime
`sudo certbot renew --dry-run`	Test that SSL renewal will work
`df -h`	Check remaining disk space
`free -h`	Check available memory
`curl ifconfig.me`	Print this server's public IP address

Common errors and how to fix them

Symptom	Cause	Fix
port is already allocated	Another process is using that port on the host.	Change the left number in the ports mapping, e.g. 3001:80. Or find the culprit with **sudo lsof -i :3000**.
permission denied ... docker daemon socket	Your user is not in the docker group yet, or the session predates the change.	Run **sudo usermod -aG docker $USER**, then log out and SSH back in.
Container status is **Restarting**	The app crashes immediately on startup.	**docker compose logs backend**. Almost always a missing .env variable or a wrong start command.
502 Bad Gateway from Nginx	Nginx is running but nothing is answering on the port it forwards to.	Check **docker ps**. Confirm the port in proxy_pass matches the left-hand port in docker-compose.yml.
Domain still shows the old page	DNS has not propagated, or your browser cached it.	Check dnschecker.org. Try an incognito window or a different network.
Certbot: **challenge failed**	The domain does not resolve to this server yet, or port 80 is blocked.	Verify DNS first, then **sudo ufw allow 'Nginx Full'** and check your provider's firewall panel.
Build dies with **Killed**	The server ran out of memory during the frontend build.	Add 2 GB of swap (see Step 12), or build the image on your laptop and push it to a registry.
Frontend loads, API calls fail	The frontend is calling localhost, which means the visitor's own machine.	Use relative paths like **/api/users** and let Nginx route them, as configured in Step 16.
Refreshing a page gives **404**	The static server is looking for a file that does not exist.	Add the **try_files** line to frontend/nginx.conf (Step 3).

Final deployment checklist

☐	Both Dockerfiles build without errors on the laptop
☐	docker compose up works locally and the app loads in the browser
☐	.gitignore excludes .env and node_modules
☐	Code is pushed to GitHub and .env is NOT in the repository
☐	Docker and Git are installed on the VPS
☐	Repository is cloned and .env recreated on the server
☐	docker ps shows every container as Up
☐	DNS A records for @ and www point to the server IP
☐	dnschecker.org shows green ticks worldwide
☐	nginx -t passes and the site is enabled
☐	http://yourdomain.com loads the app
☐	https://yourdomain.com shows the padlock
☐	certbot renew --dry-run completes successfully

Social Eagle | Gen AI Architect Program
Keep this guide beside you for your first few deployments.